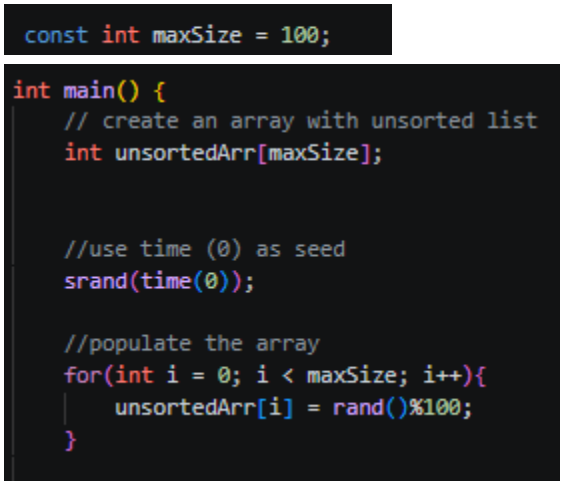
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/02/26	**DATE SUBMITTED:**	09/02/26
NAME:	Gabriel Dwyane D. Arididdon	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO 7.1 **Hands-on Activity 7.1 Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	Since it is a random generated array, every time we run the code, it displays random values that is limited to the range 0-99' due to rand() %100.

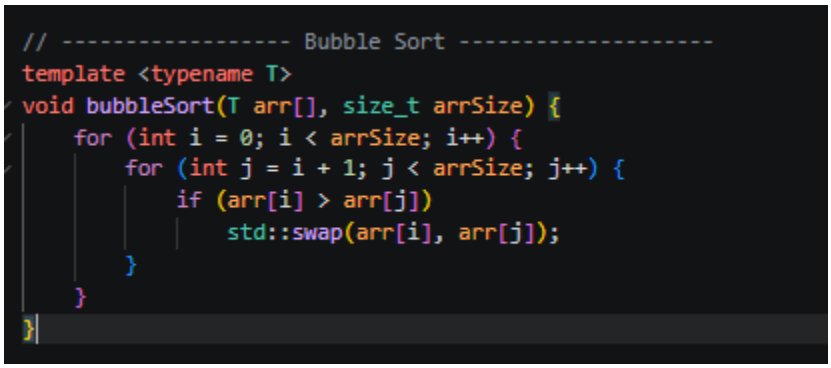
```
const int maxSize = 100;

int main() {
    // create an array with unsorted list
    int unsortedArr[maxSize];


    //use time (0) as seed
    srand(time(0));

    //populate the array
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
// ------------------ Bubble Sort --------------------
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (int i = 0; i < arrSize; i++) {
        for (int j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j])
                std::swap(arr[i], arr[j]);
        }
    }
}
```

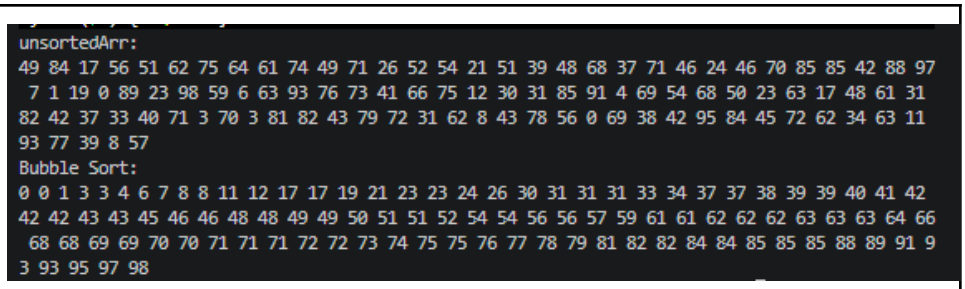
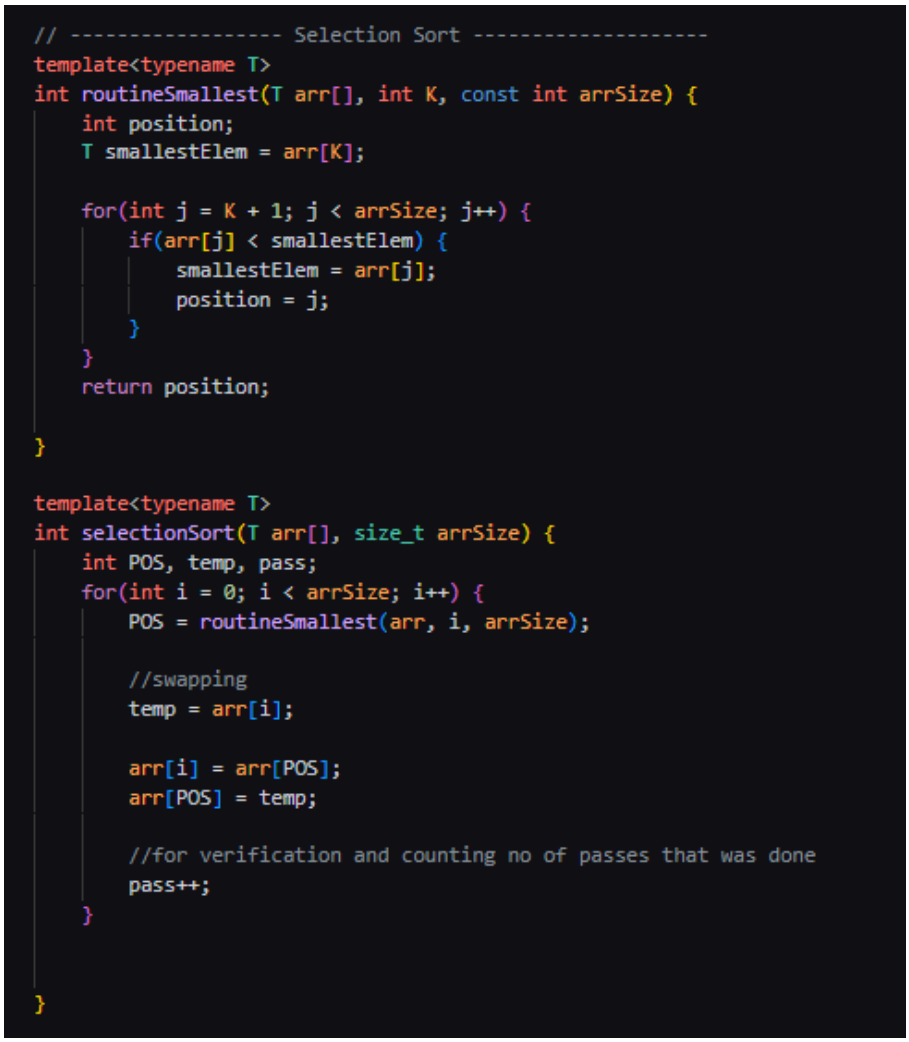
	
Observations	In BubbleSort, it goes through every values then it swaps adjacent numbers if they are out of order. Large numbers goes to the end of the array one by one. The best and worst time complexity is O(N^2) because it has outer and inner loop and it checks every index regardless if it is sorted or not.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

```cpp
// ------------------ Selection Sort --------------------
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position;
    T smallestElem = arr[K];

    for(int j = K + 1; j < arrSize; j++) {
        if(arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;

}

template<typename T>
int selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for(int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);

        //swapping
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        //for verification and counting no of passes that was done
        pass++;
    }


}
```

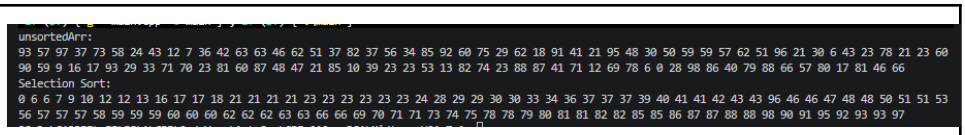
	
Observations	In SelectionSort, it divides the array into sorted and unsorted regions. It searches the entire unsorted region for the minimum element each iteration. The best and worst time complexity of this sorting algorithm is O(N^2) since it always scans all remaining items to ensure it found the smallest number among them.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observations	In InsertionSort, it is like choosing a card in a deck. It takes one number at each pass and puts it on one position until it reaches its correct spot in the array. The best case is O(N) while worst case is O(N^2) since if the left item is smaller it stops early otherwise it must slide past all previously sorted items to reach the index 0.

Table 7-4. Insertion Sort Algorithm

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

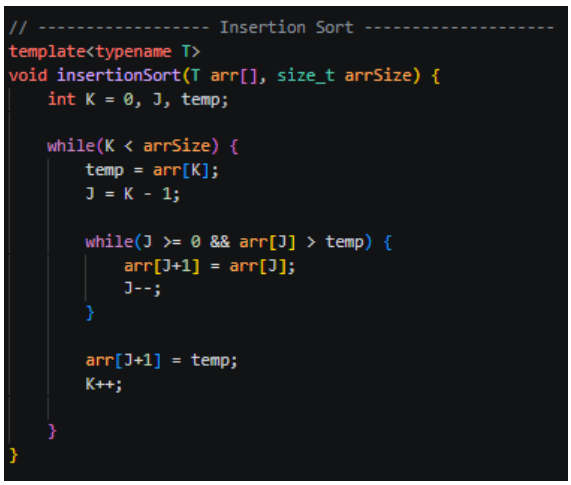
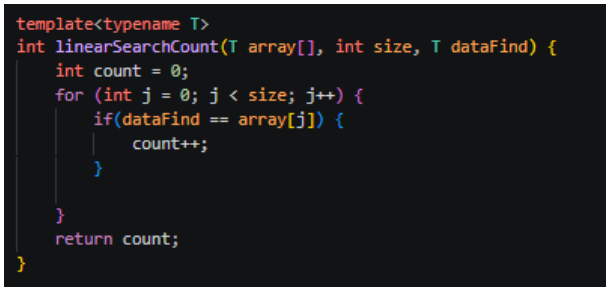
Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Pseudocode	START Set up 5 candidate names and an empty list for 100 votes // 1. Generate Votes FOR each of the 100 array positions: Assign a random vote number between 1 and 5 // 2. Display and Sort PRINT unsorted vote list SORT votes from low to high using Insertion Sort PRINT sorted vote list // 3. Count Votes FOR each candidate ID (1 through 5): COUNT how many times that ID appears in the array using Linear Search STORE the total count for that candidate // 4. Determine Winner SET highestVotes = -1 SET winner = -1 FOR each candidate: PRINT candidate and their vote count IF candidate's votes > highestVotes THEN UPDATE highestVotes = candidate's votes UPDATE winner = current candidate END IF PRINT the winning candidate and their total votes

	END

Table 7-5. Pseudocode of Algorithm

Code + Console Screenshot	 

```
// ------------------ Insertion Sort --------------------
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    int K = 0, J, temp;

    while(K < arrSize) {
        temp = arr[K];
        J = K - 1;

        while(J >= 0 && arr[J] > temp) {
            arr[J+1] = arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;

    }
}
```

```
template<typename T>
int linearSearchCount(T array[], int size, T dataFind) {
    int count = 0;
    for (int j = 0; j < size; j++) {
        if(dataFind == array[j]) {
            count++;
        }

    }
    return count;
}
```

```cpp
const int maxSize = 100;
const int num_candidates = 5;

void displayArr(int arr[]);

int main() {
    std::string candidates[num_candidates] = {
        "Candidate 1",
        "Candidate 2",
        "Candidate 3",
        "Candidate 4",
        "Candidate 5" };

// create an array with unsorted list
    int votes[maxSize];


// use time (0) as seed
    srand(time(0));

    for(int i = 0; i < maxSize; i++){
        votes[i] = (rand()% num_candidates) + 1;
    }

    std::cout << "Unsorted Votes: \n";
    displayArr(votes);

    std::cout << "Sorted Votes: \n";
    insertionSort(votes, maxSize);
    displayArr(votes);

    std::cout << std::endl;

    //-----------------linear search------------
    std::cout << "---- Election Results ---- " << std::endl;

    int voteCount[num_candidates] = {};

    for(int i = 0; i < num_candidates; i++) {

        voteCount[i] = linearSearchCount(maxSize, i, votes);
    }

    int maxVotes = -1;
    int winner = -1;
    for(int i = 0; i < num_candidates; i++) {
        std::cout << candidates[i] << ": " << voteCount[i] << " votes" <<  std::endl;

        if (voteCount[i] > maxVotes) {
            maxVotes = voteCount[i];
            winner = i;
        }
    }
    std::cout << std::endl;
    std::cout << "WINNER: " << candidates[winner] << " with " << maxVotes << " votes!\n";

    return 0;
}
```

Observations	I used the insertionsort algorithm from previous task since it has O(N) as its best time complexity compared to the other two. It also has adaptive nature that allows it to terminate inner loops early especially when processing heavily duplicated values (1-5). I used linearsearch for counting the votes for each candidate. For declaring winners, the searching algorithm counts the frequency of each candidate (1-5) then it will be stored to the voteCounts array. If it exceeds the maxVotes, then that candidate will be declared as the winner.

Table 7-6. Insertion Algo + Linear Search Screenshot

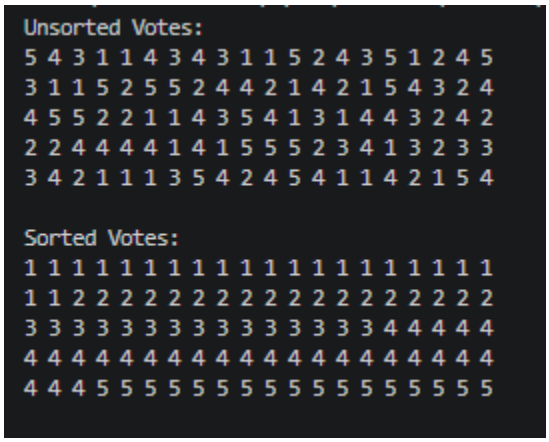
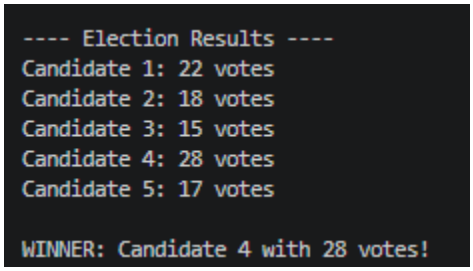
Output Console Showing Sorted Array	
Manual Count	C1 = 22 C2 = 18 C3 = 15 C4 = 28 C5 = 17
Count Result of Algorithm	

Table 7-7 Output Testing

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

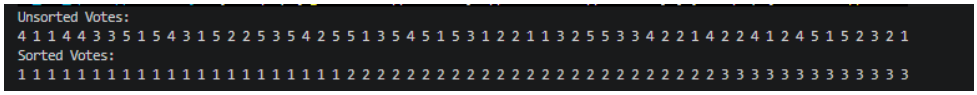
Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

I used it in the displayArr since the output doesn't display all the 100 sorted votes in the terminal



Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.